Projet E5 5SDWANA : compte rendu du groupe MIP

Groupe	MAHAUT Bryan, IRSUTTI Axel, PEIFFER Axel
Formation	ESTIAM Metz, 5e année CCSN, module 5SDWANA
Empreinte	`MIP-04-11-2026`
Dépôt Git	https://github.com/TeBryy/projet-e5-SDWAN-mip
Images Docker Hub	`axelirsutti/stock-mip-04-11-2026`, `axelirsutti/vitrine-mip-04-11-2026`, `axelirsutti/paiement-mip-04-11-2026`

1. Contexte et objectifs

Le client est un site e-commerce avec beaucoup de trafic. Il veut un MVP sur Kubernetes avec 3 applications conteneurisées, dont une pour le paiement avec Stripe. Tout doit être déployé « as code » et l'application doit pouvoir monter en charge puis redescendre.

On a fait une petite boutique fictive, « Comptoir Lorrain », avec 3 applis : vitrine, le site qui affiche les produits, paiement, qui gère le paiement avec Stripe, et stock, une API qui donne les produits et le stock. vitrine et paiement sont accessibles de l'extérieur, stock seulement depuis le cluster.

2. Choix techniques

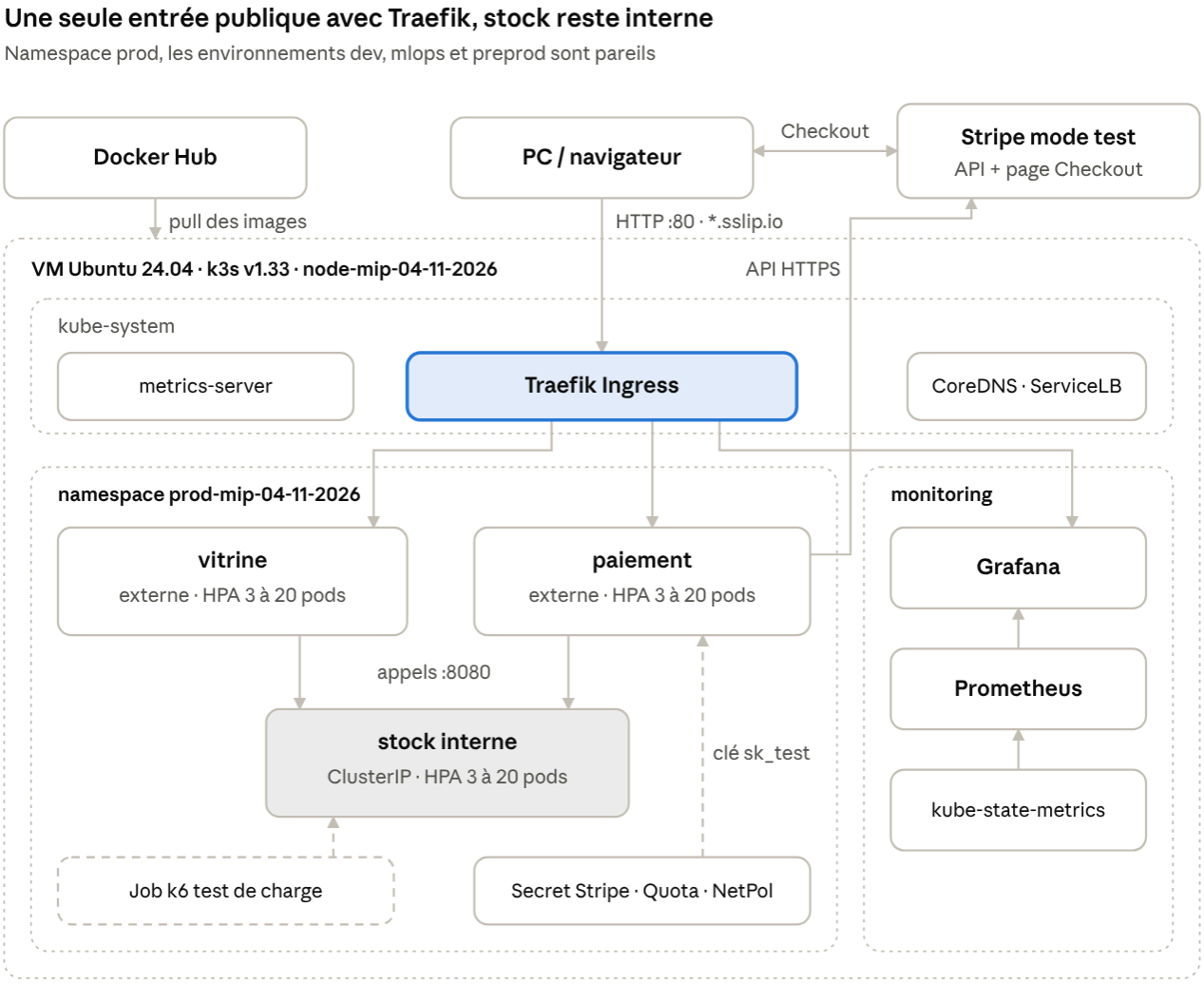
On a comparé nos choix avec ce qui est vu en cours.

Besoin	Notre choix	Vu en cours	Avantages	Inconvénients
Kubernetes	k3s	Minikube	léger, tourne sur une seule VM, Traefik, metrics-server et les NetworkPolicy sont déjà inclus	un seul nœud, donc pas de haute disponibilité

Besoin	Notre choix	Vu en cours	Avantages	Inconvénients
Langage	Go	Python / Flask	images très petites, démarrage rapide, peu de RAM	il faut compiler, l'appel à Stripe est codé à la main
Images	build en 2 étapes, image `scratch`	image Python complète	pas de shell dans l'image, donc moins de failles	plus dur à déboguer
Déploiement	Kustomize	un dossier de manifests par environnement	les 4 environnements sont identiques, on modifie un seul endroit	un peu long à prendre en main
Exposition	Ingress Traefik et sslip.io	NodePort / port-forward	une seule IP et le port 80 pour toutes les URL	dépend du DNS public sslip.io
Test de charge	k6 lancé dans le cluster	Apache Bench	on peut aussi tester stock, qui n'est pas accessible de l'extérieur	k6 tourne sur le même nœud que les applis
Supervision	Prometheus et Grafana	Netdata	on voit les pods des HPA et le CPU, le dashboard est dans Git	plus lourd en RAM
CI	GitHub Actions	build à la main	tests et scan Trivy avant de pousser les images	il faut configurer les secrets Docker Hub

On n'a pas pris Minikube parce qu'il est surtout fait pour tester en local et que pour l'exposer à l'extérieur il faut passer par du port-forward. Avec k3s, les applis sont directement accessibles sur l'IP de la VM.

3. Architecture



Architecture du projet, ici le namespace prod

Tout passe par Traefik sur le port 80, qui envoie vers la bonne appli selon le nom d'hôte. stock n'a pas d'Ingress, seuls vitrine et paiement peuvent l'appeler.

Composant	Rôle
VM et k3s	un seul nœud qui fait master et worker
Traefik	Ingress Controller : reçoit le trafic HTTP sur l'IP de la VM et l'envoie vers la bonne appli
metrics-server	donne le CPU et la RAM des pods, nécessaire pour le HPA
Deployment	garde 3 pods par appli
Service ClusterIP	adresse fixe et répartition de charge entre les pods

Composant	Rôle
HPA	ajoute ou enlève des pods selon le CPU, entre 3 et 20
NetworkPolicy	filtre les connexions entre les pods
ResourceQuota	limite les ressources de chaque environnement
Job k6	génère la charge pendant le test
Prometheus et Grafana	récupèrent les métriques et les affichent

4. Répartition des tâches

On s'est réparti le projet en 3 parties.

	Bryan MAHAUT	Axel IRSUTTI	Axel PEIFFER
Partie	cluster, réseau et environnements	applications et paiement	performance et supervision
Travail	installation de k3s, appli stock, Ingress, NetworkPolicy, quotas, environnements mlops, preprod et prod	applis vitrine et paiement, Dockerfiles, images Docker Hub, Stripe, CI GitHub	mesures CPU/RAM, HPA, test de charge k6, Prometheus et Grafana

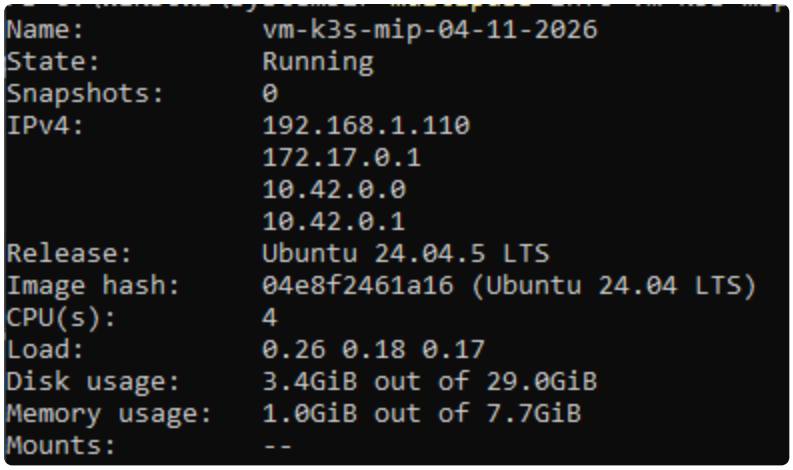
Le compte rendu a été écrit ensemble et le code est sur un dépôt GitHub commun.

5. Partie 1

5.1 Création de la VM

On a créé la VM avec Multipass et VirtualBox : 4 vCPU, 8 Go de RAM, 30 Go de disque et une carte réseau en mode pont.

```
multipass launch 24.04 --name vm-k3s-mip-04-11-2026 --cpus 4 --memory 8G --
disk 30G --network Ethernet
multipass shell vm-k3s-mip-04-11-2026
export HISTTIMEFORMAT="%F %T "
git clone https://github.com/TeBryy/projet-e5-SDWAN-mip.git
```

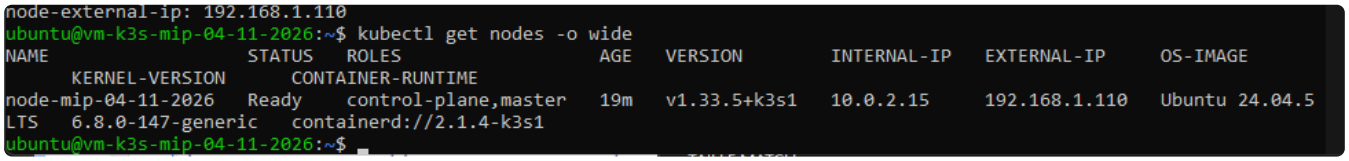


Capture 1 : la VM avec l'IP 192.168.1.110 sur le réseau local.

5.2 Installation de k3s

La configuration de k3s est dans `infra/k3s-config.yaml`. On a mis 250 pods maximum pour le test de charge, gardé des ressources pour le système et activé le chiffrement des secrets.

```
sudo ./infra/install-k3s.sh 192.168.1.110
kubectl get nodes -o wide
```



Capture 2 : le nœud est Ready avec l'IP 192.168.1.110.

5.3 Les applications

Appli	Routes	Rôle
stock	`GET /api/produits`, `GET /api/produits/{id}`	renvoie les 5 produits et leur stock en JSON
vitrine	`GET /`	affiche les produits récupérés depuis stock
paiement	`GET /`, `POST /checkout`, `GET /succes`, `GET /annule`	crée la session de paiement Stripe

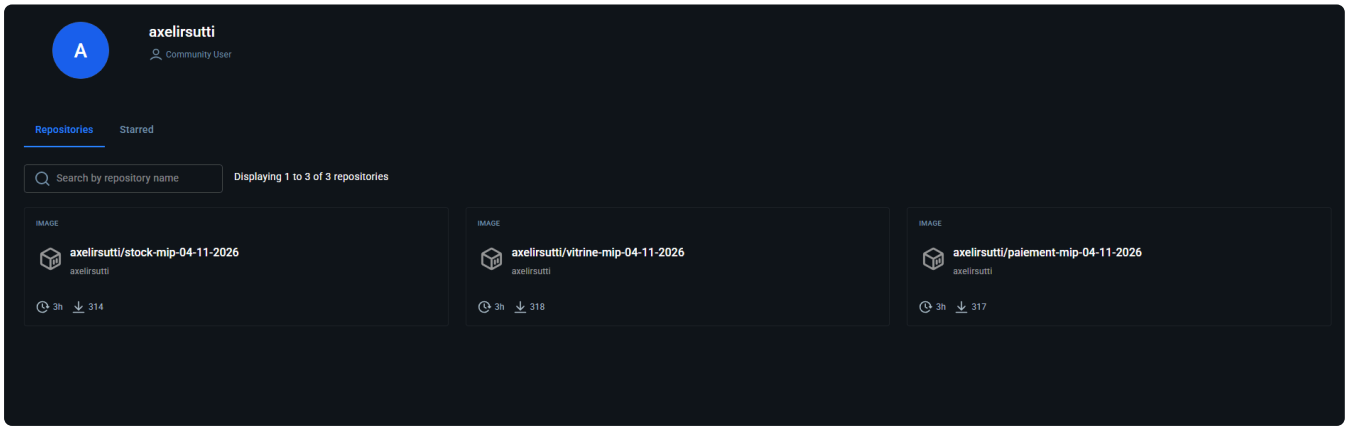
Les 3 applis ont `/healthz` et `/readyz` pour les sondes Kubernetes. Elles renvoient aussi le nom du pod dans l'en-tête `X-Servi-Par`, ce qui permet de voir que les requêtes sont réparties sur les 3 réplicas. Pour le paiement, le prix est repris depuis stock et pas depuis

le navigateur.

5.4 Images Docker

Le Dockerfile est en 2 étapes : on compile le Go, puis on garde seulement le binaire dans une image `scratch` avec un utilisateur non root. Les images font entre 9 et 13 Mo.

Les images sont construites et poussées sur Docker Hub par GitHub Actions à chaque push sur `main` : tests Go, vérification des manifests, build, scan Trivy, puis push.



Capture 3 : les 3 images sur Docker Hub.

5.5 Mesures avant de fixer les limites

Avant d'écrire les `requests` et `limits`, on a lancé les images avec docker et envoyé du trafic avec k6 à 20 puis 200 requêtes/s en regardant `docker stats`. Le détail est dans `docs/mesures-ressources.md`.

Appli	RAM au repos	RAM à 200 req/s	CPU à 200 req/s	requests	limits
vitrine	1,9 Mi	8,7 Mi	200m	50m / 16Mi	200m / 64Mi
stock	1,8 Mi	7,0 Mi	80m	25m / 16Mi	100m / 48Mi
paiement	1,9 Mi	7,7 Mi	90m	25m / 16Mi	150m / 64Mi

Le request correspond à un trafic normal et sert au calcul du HPA. La limite CPU est environ 4 fois plus haute pour tenir un pic, et on a laissé de la marge sur la RAM.

5.6 Manifests et déploiement en dev

Les manifests communs sont dans `k8s/base/` : un Deployment et un Service par appli, les HPA, les Ingress de vitrine et paiement, les NetworkPolicy et les quotas.

```
./scripts/configure.sh
echo "STRIPE_SECRET_KEY=sk_test_..." > .env.stripe
./scripts/deploy.sh dev
kubectl get deploy,hpa,ingress -n dev-mip-04-11-2026
```

Le script crée le namespace et le secret Stripe, applique les manifests et attend que les pods soient prêts. Les URL utilisent sslip.io, qui renvoie vers l'IP de la VM, donc pas besoin de modifier le fichier hosts.

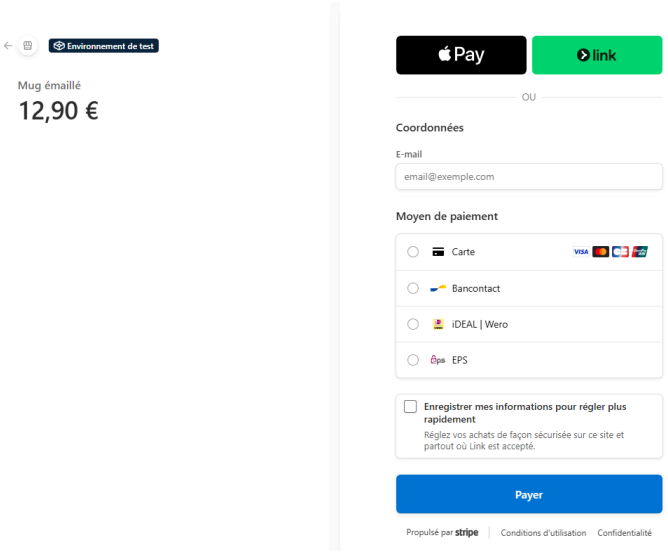


Capture 4 : la vitrine en dev sur `http://vitrine.dev.192.168.1.110.sslip.io`.

5.7 Test de paiement Stripe

1. Création d'un compte Stripe en mode test, la clé `sk_test_` est mise dans `.env.stripe`.
2. Sur la vitrine, clic sur Acheter : on est redirigé vers la page Stripe Checkout.
3. Paiement avec la carte de test `4242 4242 4242 4242`.
4. Retour sur la page de succès avec le statut `paid`.

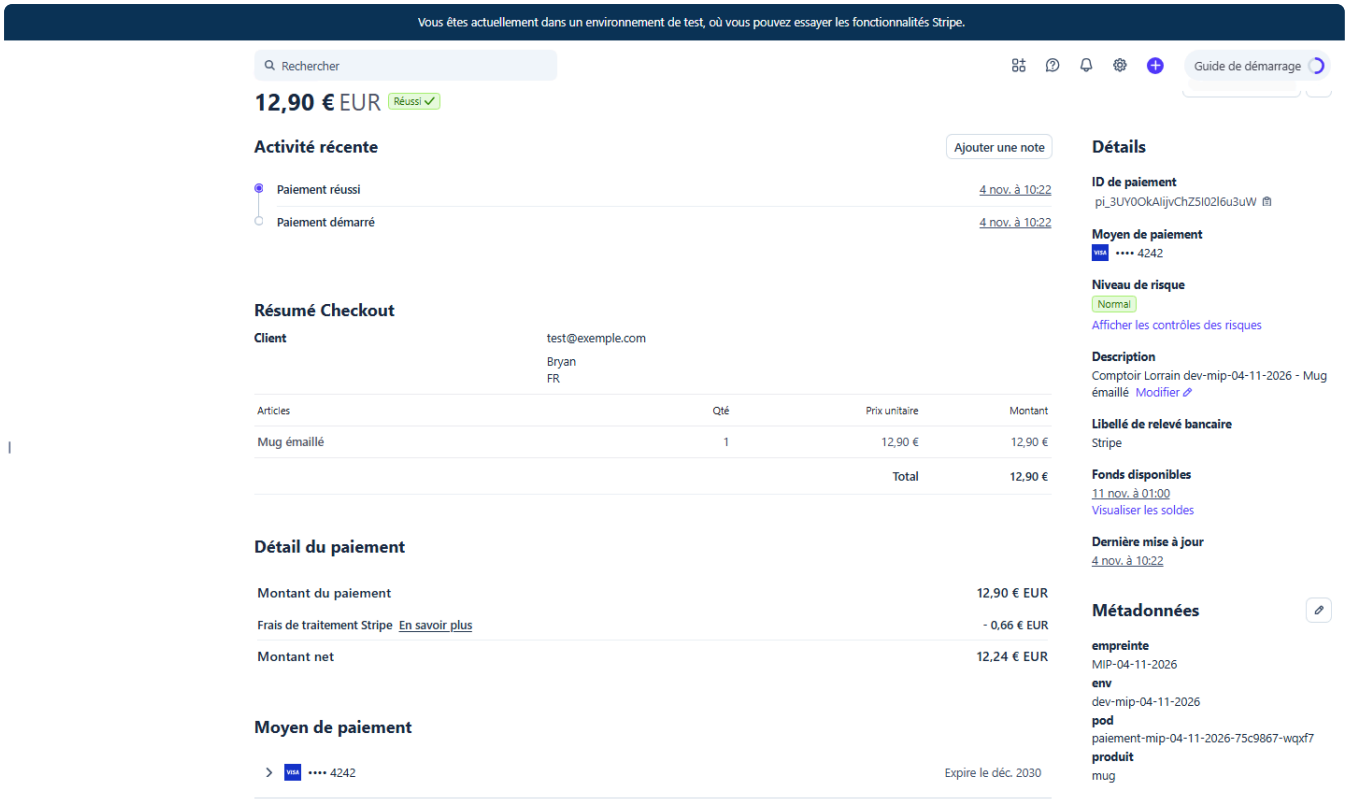
Le paiement apparaît dans le dashboard Stripe avec les métadonnées `env`, `pod` et `empreinte`, ce qui permet de savoir de quel environnement et de quel pod il vient.



Capture 5 : la page Stripe Checkout.



Capture 6 : la page de succès après le paiement.



Capture 7 : le paiement de 12,90 € dans le dashboard Stripe.

5.8 Autoscaling

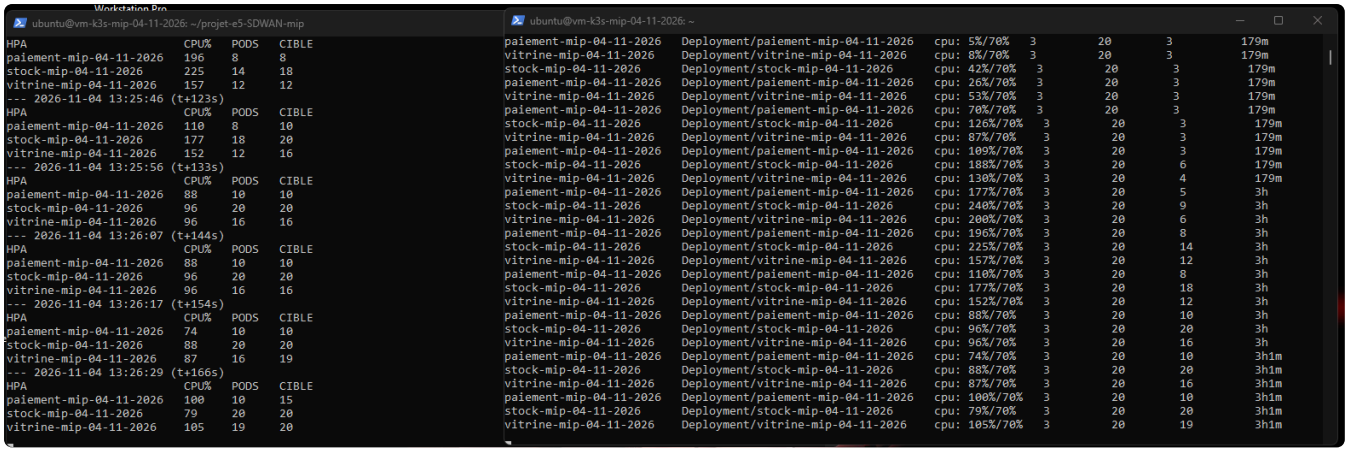
```
minReplicas: 3
maxReplicas: 20
metrics:
  - type: Resource
    resource: { name: cpu, target: { type: Utilization, averageUtilization:
70 } }
behavior:
  scaleDown:
    stabilizationWindowSeconds: 60
```

Le HPA se base sur le CPU avec une cible de 70 %. On a mis 60 secondes avant de redescendre au lieu de 5 minutes, pour voir le retour à 3 pods plus vite.

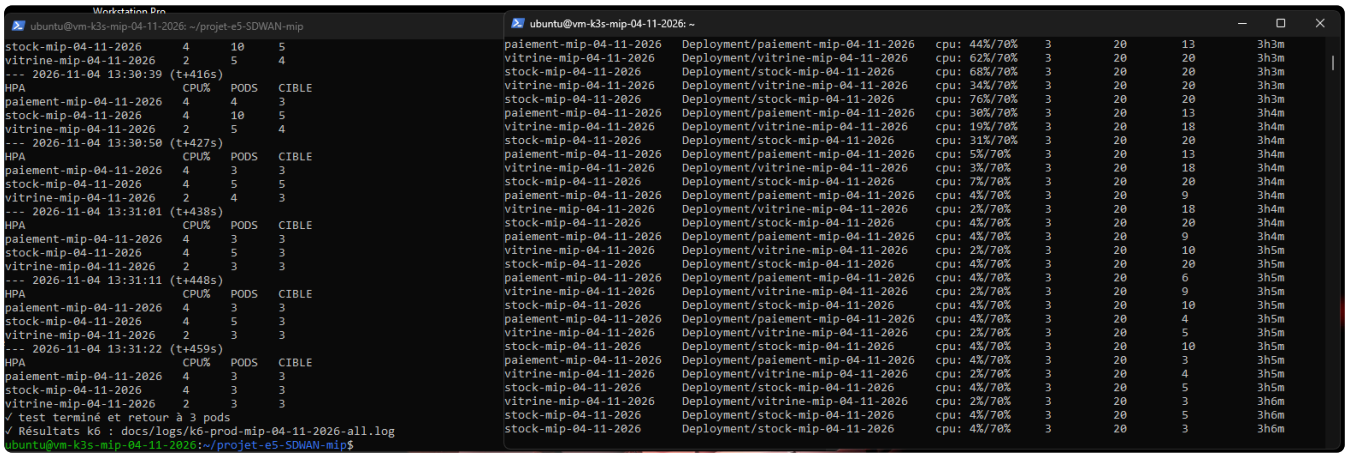
5.9 Test de charge

Le test est un job k6 lancé dans le namespace prod. Il monte jusqu'à 800 requêtes/s par appli, garde ce niveau pendant 3 minutes puis redescend. Sur paiement on teste seulement la page d'accueil, pour ne pas créer des milliers de sessions Stripe.

```
./scripts/test-charge.sh prod all 800
kubectl get hpa -n prod-mip-04-11-2026 -w
```



Capture 8 : pendant le test, les pods montent jusqu'à 20.



Capture 9 : après le test, retour à 3 pods par appli.

Les résultats sont dans la partie 9.

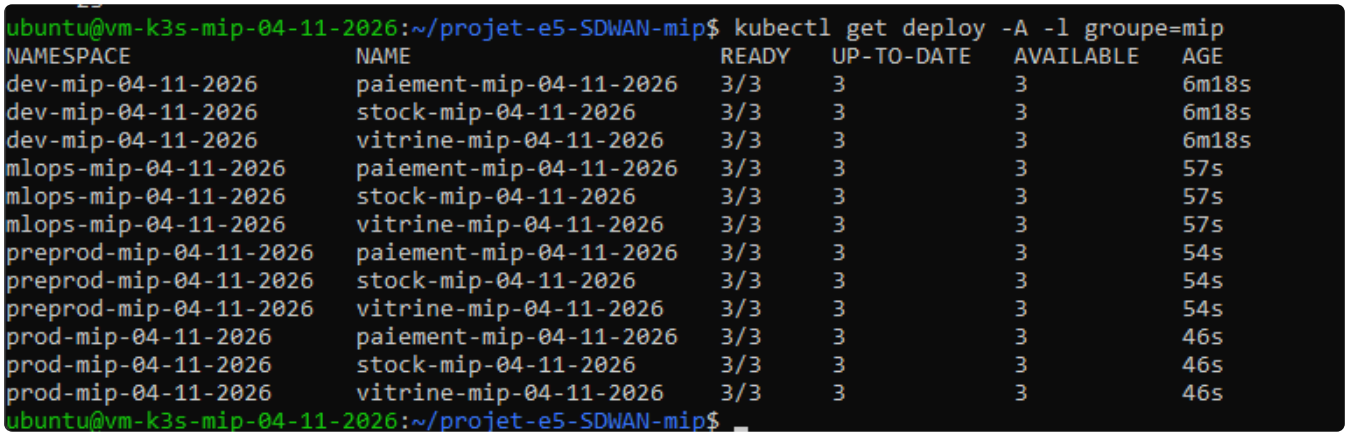
6. Partie 2 : environnements mlops, preprod et prod

Chaque environnement a son namespace. Ils sont tous créés à partir de la même base Kustomize, seuls le namespace et les noms d'hôte changent.

```
k8s/
├── base/          deployments, services, hpa, ingress, networkpolicy, quotas
├── components/    copie les noms d'hôte dans les Ingress
└── overlays/
    ├── dev/
    ├── mlops/
    ├── preprod/
    └── prod/
```

```
./scripts/deploy.sh mlops preprod prod
kubectl get deploy -A -l groupe=mip
```

Mécanisme	Ce que ça isole
Namespace	chaque environnement a ses propres pods, services et secrets
ResourceQuota	un environnement ne peut pas prendre toutes les ressources du nœud
NetworkPolicy	un pod d'un autre environnement ne peut pas appeler stock
Pod Security	les pods root ou privilégiés sont refusés



Capture 10 : les 12 Deployments en 3/3.

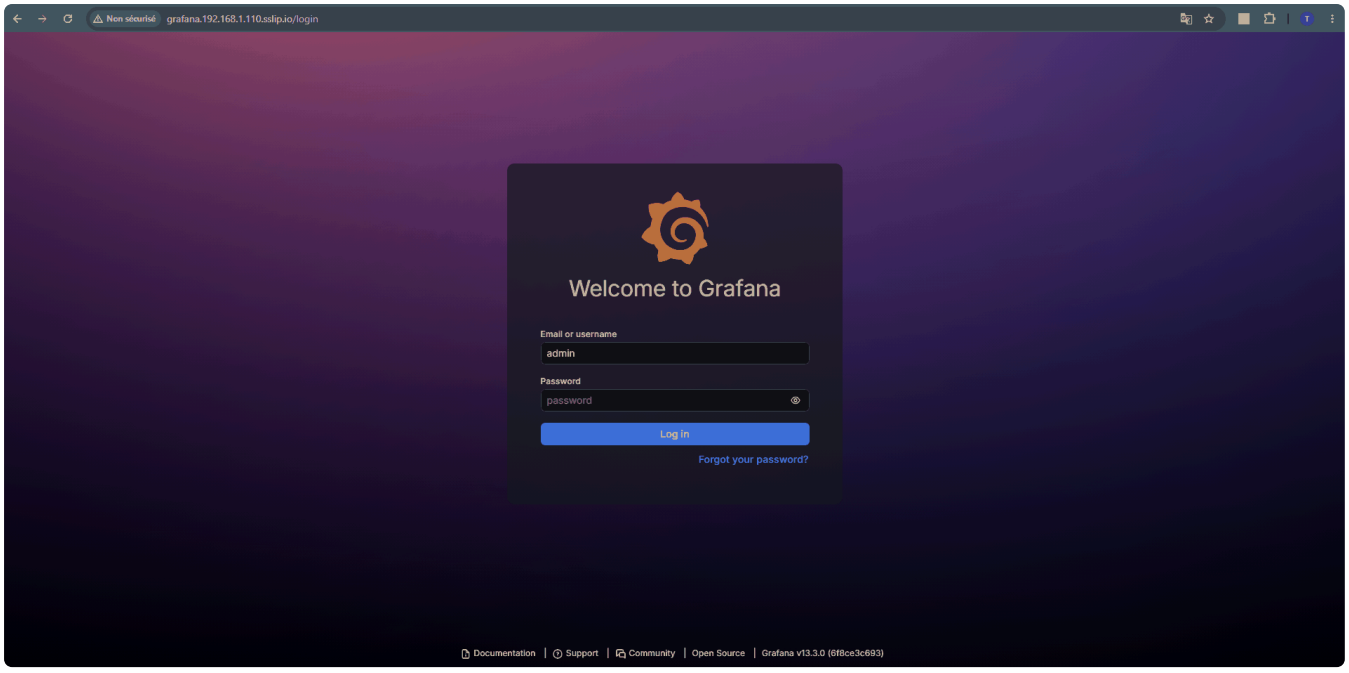
7. Partie 3 : supervision

On a installé kube-prometheus-stack avec Helm dans le namespace `monitoring-mip-04-11-2026`. Grafana est accessible sur `http://grafana.192.168.1.110.sslip.io`. On a désactivé Alertmanager et les cibles que k3s ne permet pas de surveiller séparément.

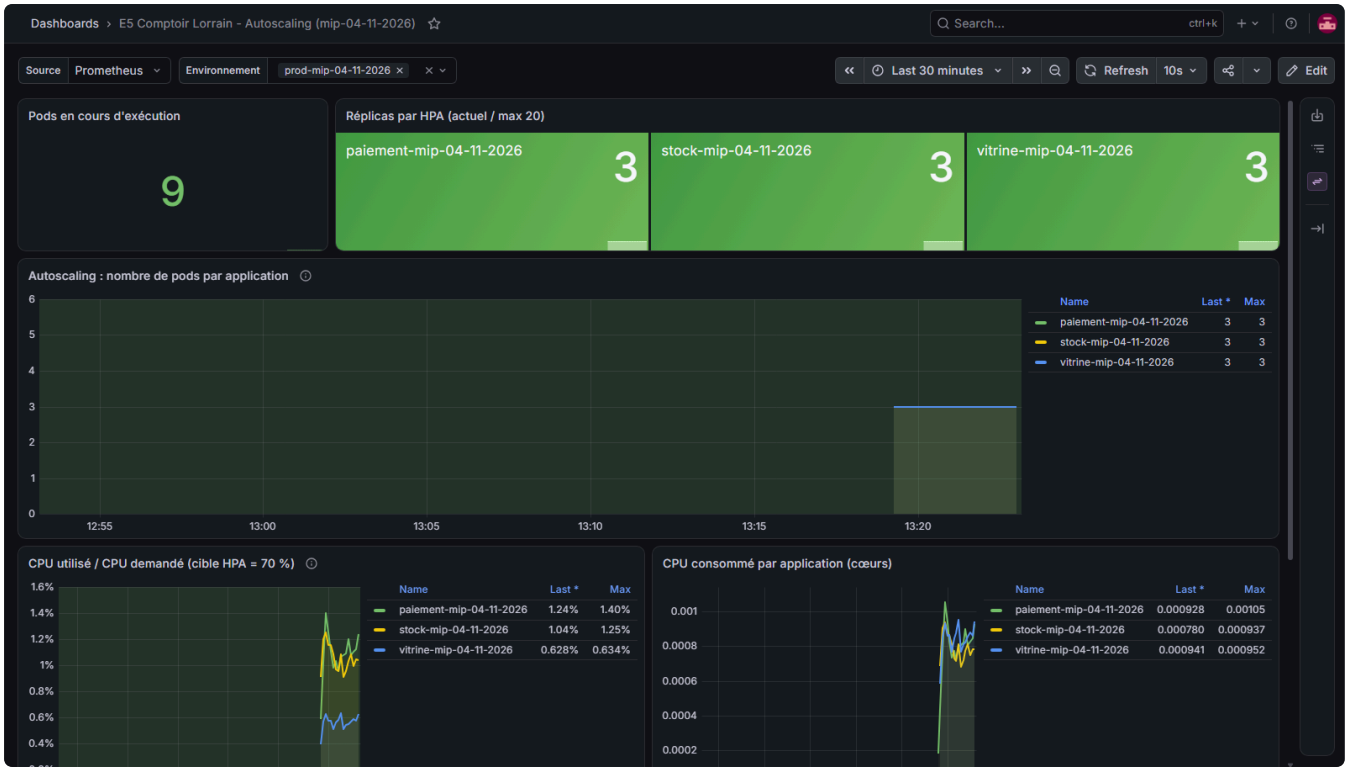
```
./observability/install.sh
kubectl get pods -n monitoring-mip-04-11-2026
```

Au début Grafana redémarrait en boucle : la limite mémoire de 256Mi était trop basse, on l'a passée à 512Mi.

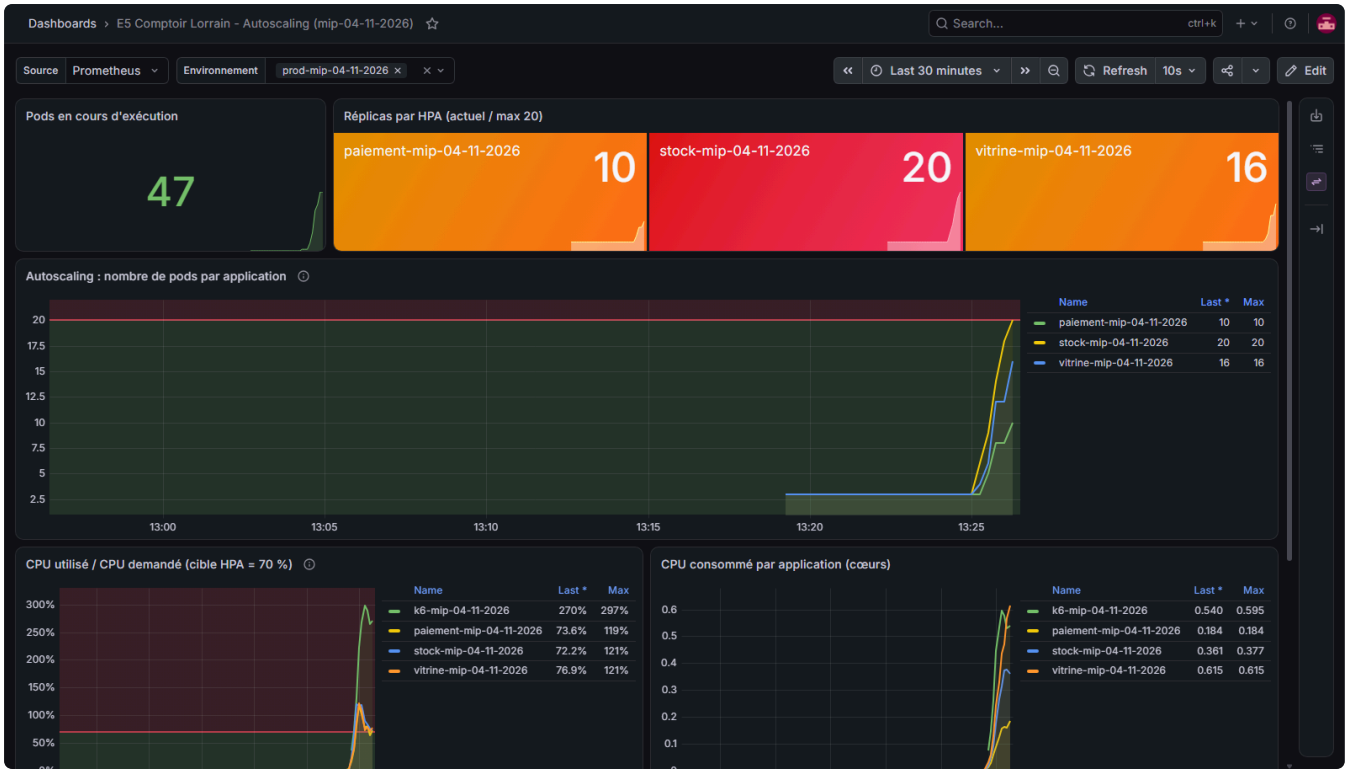
On a fait un dashboard « E5 Comptoir Lorrain - Autoscaling » avec le nombre de pods, les réplicas de chaque HPA, le CPU et la RAM par appli. Il est dans `observability/dashboard-autoscaling.json`.



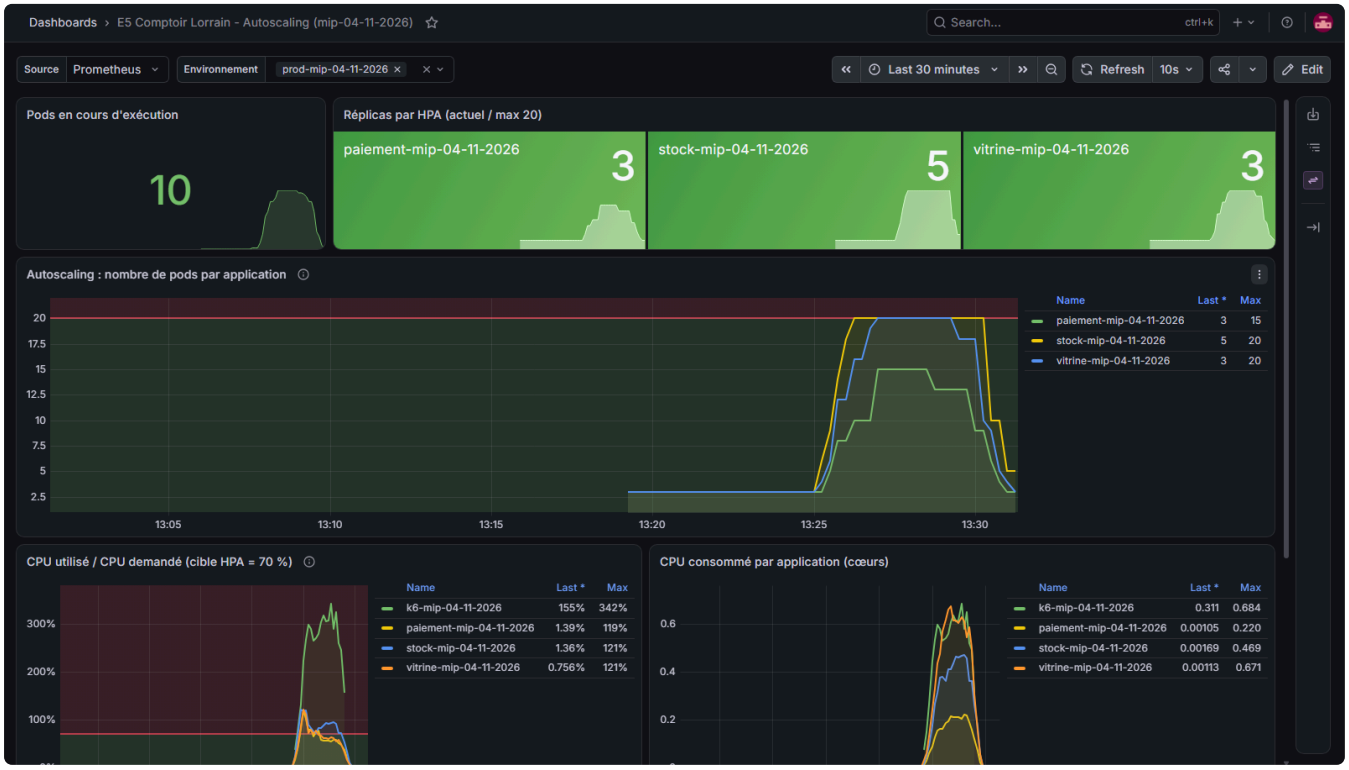
Capture 11 : Grafana accessible depuis le navigateur.



Capture 12 : avant le test, 3 pods par appli.



Capture 13 : pendant le test, 47 pods et le CPU au-dessus de 70 %.



Capture 14 : la courbe complète, montée jusqu'à 20 puis redescente.

8. Sécurité

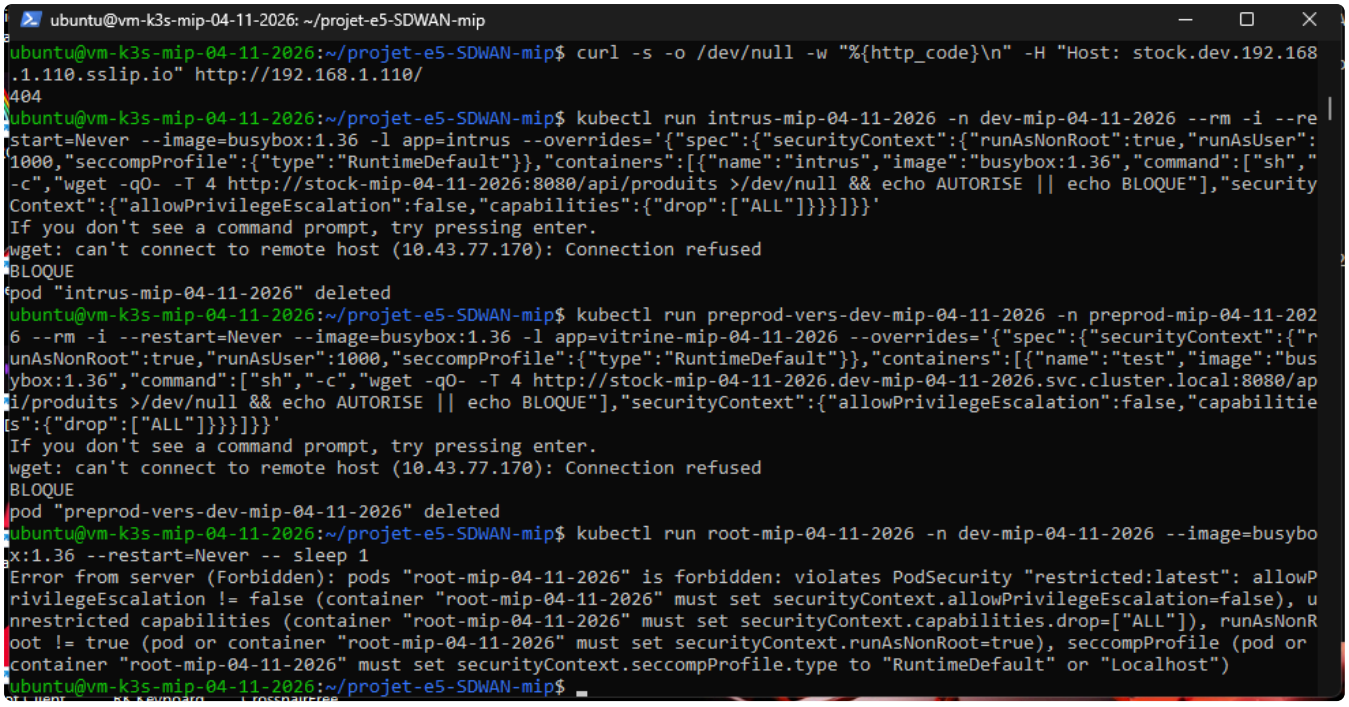
Les images sont en `scratch`, sans shell, et tournent avec un utilisateur non root. Les namespaces sont en Pod Security `restricted`, donc un pod root est refusé. Les NetworkPolicy bloquent tout par défaut, puis on autorise seulement Traefik vers vitrine et paiement, et vitrine et paiement vers stock.

La clé Stripe est dans un Secret Kubernetes et l'appli paiement refuse de démarrer avec une vraie clé. Le prix est repris depuis stock et pas depuis le navigateur. La CI fait un scan Trivy avant de pousser les images, et les NodePort sont interdits par le quota pour que tout passe par Traefik.

Tests faits sur le cluster :

```
# stock n'est pas accessible depuis l'extérieur
curl -s -o /dev/null -w "%{http_code}\n" -H "Host:
stock.dev.192.168.1.110.sslip.io" http://192.168.1.110/
# pod sans autorisation vers stock
kubectl run intrus-mip-04-11-2026 -n dev-mip-04-11-2026 --rm -i --
restart=Never --image=busybox:1.36 -l app=intrus ...
# pod de preprod vers le stock de dev
kubectl run preprod-vers-dev-mip-04-11-2026 -n preprod-mip-04-11-2026 --rm -i
--restart=Never --image=busybox:1.36 ...
# pod root
kubectl run root-mip-04-11-2026 -n dev-mip-04-11-2026 --image=busybox:1.36 --
restart=Never -- sleep 1
```

Test	Résultat
stock via Traefik	404
pod non autorisé vers stock	bloqué
pod de preprod vers le stock de dev	bloqué
pod root	refusé



Capture 15 : résultat des tests de sécurité.

9. Résultats

Test de charge en prod

Appli	Pods au départ	Pods max	CPU max	p95	Erreurs
vitrine	3	20	200 %	127 ms	2 / 161 350
stock	3	20	240 %	79 ms	2 / 171 878
paiement	3	15	196 %	78 ms	2 / 173 187

Test fait le 4 novembre 2026 entre 13h23 et 13h31. Au total 506 415 requêtes, environ 1 688 requêtes/s, 0,00 % d'erreurs. Les pods sont passés de 3 à 20 en un peu plus de 2 minutes et sont revenus à 3 environ 2 min 30 après la fin de la charge. Les 6 erreurs sont arrivées dans la première seconde du test, le temps que la NetworkPolicy prenne en compte le pod k6.

Les fichiers du test sont dans `docs/logs/` : `hpa-prod-mip-04-11-2026-all.log`, `describe-hpa-prod-mip-04-11-2026.log` et `k6-prod-mip-04-11-2026-all.log`.

Preuves demandées

Demande	Fichier
Historique des commandes Linux	`docs/logs/history-projet-MIP.log`

Demande	Fichier
Logs Docker	`docs/logs/history-projet-MIP-docker-conteneur-<appli>.log`
Describe des composants	`docs/logs/history-projet-MIP-k8s-<env>-describe.log`
Events	`docs/logs/history-projet-MIP-k8s-<env>-events.log`
Logs des pods	`docs/logs/history-projet-MIP-k8s-<env>-logs-<appli>.log`
Erreurs rencontrées	`docs/erreurs-projet-MIP.md`
Captures	`docs/captures/`

10. Difficultés et améliorations

Les erreurs rencontrées sont détaillées dans `docs/erreurs-projet-MIP.md` :

- Multipass ne démarrait pas car Hyper-V n'est pas activé sur le PC : on est passé sur VirtualBox.
- k3s prenait l'IP NAT de VirtualBox et `kubectl` affichait « permission denied » : on a ajouté `node-external-ip` et changé les droits du fichier de config.
- Le fichier `.env.stripe` était mal rempli et la clé s'est affichée dans le terminal : on a régénéré la clé et corrigé le script.
- Grafana redémarrait en boucle : limite mémoire passée de 256Mi à 512Mi.

Limites du projet :

- un seul nœud, donc si la VM tombe tout s'arrête ;
- pas de HTTPS ;
- le stock ne diminue pas vraiment après un achat ;
- pas de webhook Stripe pour confirmer le paiement.

Améliorations possibles : HTTPS avec cert-manager, webhook Stripe pour mettre à jour le stock, une vraie base de données, un cluster à plusieurs nœuds et un déploiement automatique avec Argo CD.

Conclusion

Au final tout ce qui était demandé fonctionne : les 3 applis tournent avec 3 réplicas, le paiement Stripe passe, l'autoscaling monte à 20 pods et redescend à 3, les 4 environnements sont identiques et Grafana permet de suivre tout ça. Le projet nous a surtout fait pratiquer Kustomize et le HPA, et lire les events et les describe pour trouver d'où venaient les erreurs.